

VecScaleDB: Distributed Vector Database

Architecture, Feature Validation, Dataset Evaluation,
and Benchmark Report

Team Deadlock and Chill

4th May 2026

Abstract

VecScaleDB is a distributed vector database engineered from scratch for a distributed-systems course project. It separates the concerns of metadata coordination, write ingestion, immutable segment storage, and stateless reader execution. The system uses a WAL-backed LSM-style storage engine, FAISS IVF_FLAT segment indexes, shared-storage segment files, and coordinator-driven routing with scatter-gather and replicated search modes. This report documents the system architecture, all request flows, dataset configuration, benchmark methodology, measured results, and the test strategy used to validate the features promised in the project proposal.

Contents

1	Executive Summary	3
2	System Architecture	3
2.1	Component Overview	3
2.2	Port Map	4
2.3	Architecture Diagram	4
2.4	Storage Layout	4
3	Request Flows	5
3.1	Insert and Delete Flow	5
3.2	Search Flow	5
3.3	WAL Recovery Flow	6
4	Dataset Information	6
5	Benchmark Methodology	6
5.1	Measured Quantities	6
5.2	Key Commands	6
6	Benchmark Results	7
6.1	Load Throughput	7
6.2	Search Throughput and Latency	8
6.3	Recall	9
6.4	Reader Scaling	9
7	Correctness and Feature Tests	10
7.1	Test Suite Overview	10
7.2	Key Feature Explanations	11

7.2.1	IVF_FLAT Vector Search	11
7.2.2	WAL Durability	11
7.2.3	Snapshot Isolation	11
7.2.4	Scatter-Gather vs. Replicated Modes	11
8	Fault Tolerance	12
8.1	Fault-Tolerance Matrix	12
8.2	Manual Fault-Tolerance Commands	12
9	Discussion	12
9.1	What Works	12
9.2	Observed Benchmark Behaviour	13
9.3	Known Caveats	13
10	Conclusion	13
A	Runtime Setup Reference	13
B	Key Source Paths	14
C	Health Check Commands	14

1 Executive Summary

VecScaleDB implements a WAL-backed LSM-style vector storage engine, FAISS IVF_FLAT segment indexes, shared-storage segment files, coordinator metadata routing, writer ingestion, reader-side segment caching, scatter-gather search, and replicated reader benchmark mode. A fresh proposal-level feature test suite plus a comprehensive unit and integration suite validate all core mechanisms.

Table 1: Implementation status against proposal features.

Feature	Status	Evidence
Distributed and scalable architecture	Done	Coordinator ring, writer, readers, shared storage, etcd, sharding, and 5-reader Docker topology.
Vector similarity search	Done	IVF_FLAT segment indexes with external IDs, top-k search, and nprobe control.
Dynamic data management	Done	Insert, delete (tombstones), MemTable flush, immutable segments, and reinsert-as-update semantics.
Snapshot isolation	Done	Search visibility filtered by snapshot ID across MemTable and segments.
Fault tolerance	Done	WAL recovery, shared-storage reader reload, coordinator TTL election, and coordinator-client failover.
Distributed 5-node cluster	Done	Docker Compose: 3 coordinators, 1 writer, 5 readers, etcd.

2 System Architecture

2.1 Component Overview

The cluster follows a *shared-storage* design. Compute nodes are stateless or recoverable, while durability is provided by WAL files and immutable segment files written to a host volume that all nodes can access.

Table 2: Runtime roles and responsibilities.

Role	Count	Responsibility
Coordinator	3 replicas	Maintains node registrations, segment metadata, leader state, current snapshot metadata, routing, and search fanout policy.
Writer	1 process	Accepts inserts/deletes, appends WAL records, updates the MemTable, flushes immutable segments, and registers segment metadata with the coordinator.
Reader	1–5 processes	Refreshes segment metadata, loads assigned or replicated segments from shared storage, searches its local segment cache, and returns partial top-k results to the coordinator.
Shared storage	Host volume	Stores <code>index.faiss</code> , <code>ids.npy</code> , <code>vectors.npy</code> , <code>meta.json</code> , <code>tombstones.json</code> , and WAL files.
etcd	1 service	Provides a metadata store and coordinator leader election via TTL campaigns.

2.2 Port Map

Table 3: Default port assignments in distributed Docker mode.

Service	Port	Key endpoints
Coordinator 0	8000	/search, /segments, /health, /metrics
Coordinator 1	8001	Same as coordinator 0 (replica)
Coordinator 2	8002	Same as coordinator 0 (replica)
Writer 0	8100	/insert, /delete, /health, /metrics
Reader 0	8200	/ready, /metrics
Reader 1	8201	/ready, /metrics
Reader 2	8202	/ready, /metrics
Reader 3	8203	/ready, /metrics
Reader 4	8204	/ready, /metrics

2.3 Architecture Diagram

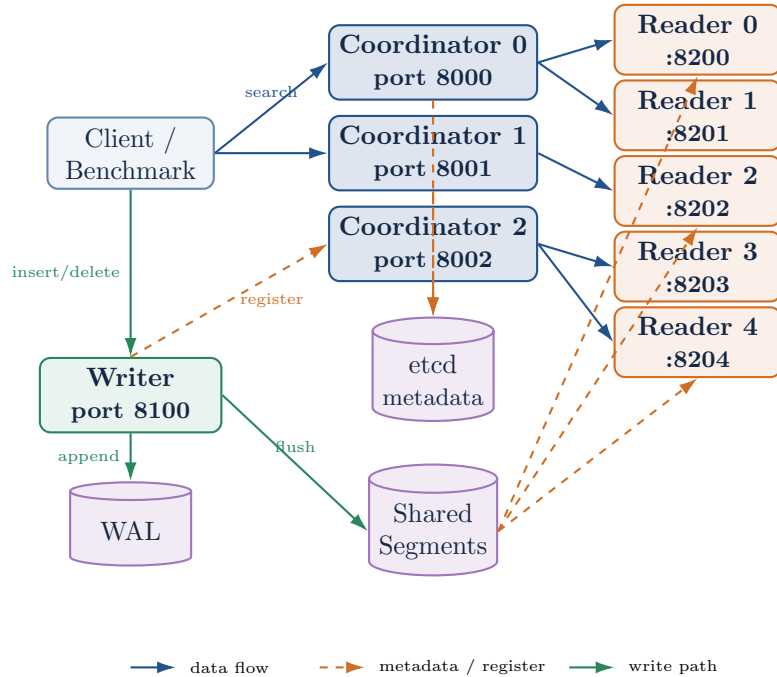


Figure 1: VecScaleDB shared-storage architecture. Coordinators maintain metadata and fan out search queries. The writer persists mutations and creates immutable segments. Readers load segments on demand for search execution.

2.4 Storage Layout

Each flushed segment is immutable and self-contained. The on-disk layout is:

- **index.faiss** — serialised FAISS IVF_FLAT index (or `IndexFlatL2` for tiny segments).
- **ids.npy** — external vector IDs as a NumPy array.
- **vectors.npy** — raw float32 vectors used for merge/compaction.
- **meta.json** — segment ID, shard ID, vector count, snapshot ID, creation timestamp, and storage path.

Deletes are stored as WAL records with operation type `OP_DELETE` and are persisted in `tombstones.json`. Both single-node search and distributed readers filter tombstoned IDs before physical merge.

3 Request Flows

3.1 Insert and Delete Flow

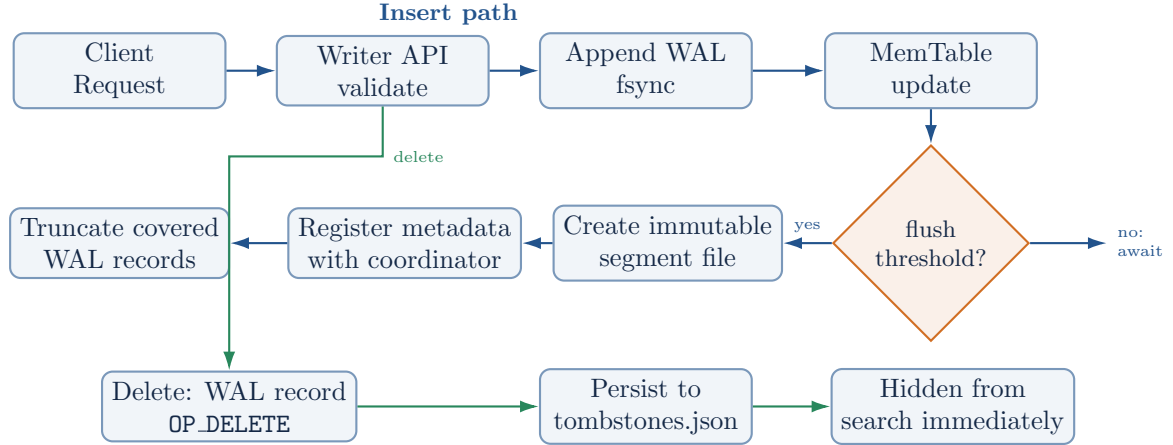


Figure 2: Write path. WAL append happens before the mutation becomes visible, ensuring durability. Threshold-triggered flushes convert MemTable contents into immutable FAISS-backed segments. Deletes are WAL records that are immediately reflected in search via tombstone filtering.

3.2 Search Flow

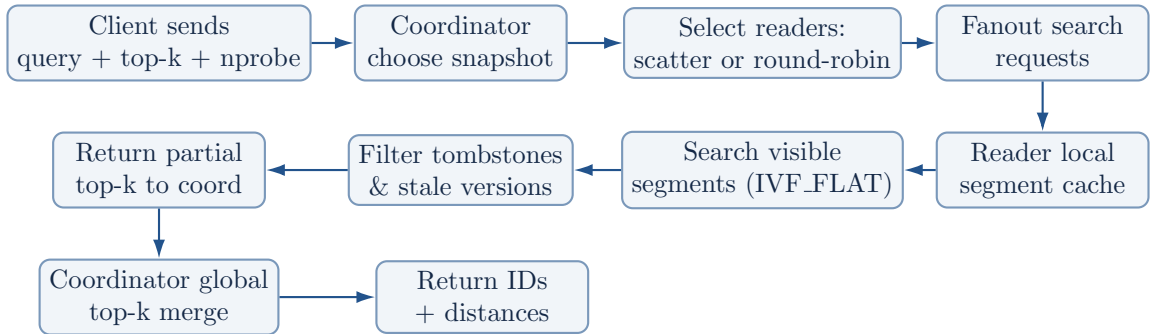


Figure 3: Search path. Queries execute against a consistent snapshot. Readers search their local FAISS segment caches and return partial top-k results that the coordinator merges into the final answer.

3.3 WAL Recovery Flow

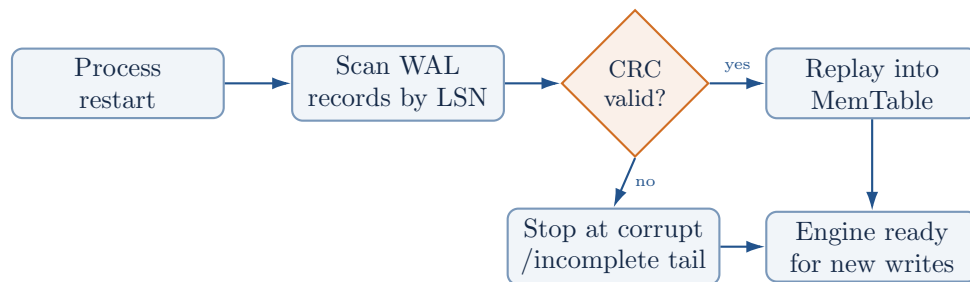


Figure 4: WAL recovery. The engine replays records newer than the latest flushed segment. Corrupt or incomplete trailing records are ignored so partial writes do not poison recovery.

4 Dataset Information

The evaluation uses the datasets specified in the project proposal. SIFT10K is derived from the first 10,000 SIFT1M training vectors for fast correctness and smoke testing. SIFT1M is the primary performance benchmark. Deep1M uses 96-dimensional Deep image vectors from the public ANN-Benchmarks dataset; its public ground truth is angular while VecScaleDB evaluates with L2.

Table 4: Datasets used in this report.

Dataset	Train	Queries	Dim	Metric	Purpose
SIFT10K	10,000	100	128	L2	Fast correctness, smoke metrics, recall sanity.
SIFT1M	1,000,000	1,000	128	L2	Primary performance and reader-scaling benchmark.
Deep1M	1,000,000	1,000	96	L2	Alternate dataset; note angular ground truth.

Note on Deep1M recall. The publicly available Deep1M ground truth is computed with cosine/angular distance. Since VecScaleDB uses L2, recall evaluation for Deep1M is skipped by default. QPS and latency measurements are still collected.

5 Benchmark Methodology

5.1 Measured Quantities

Each benchmark run collects:

- load time and vectors inserted per second,
- total completed search queries, error count, and QPS,
- p50, p95, and p99 search latency (ms),
- recall@10 where compatible ground truth exists,
- QPS-vs-reader-count for 1–5 readers (scaling evaluation).

5.2 Key Commands

Listing 1: Run full automated test suite.

```

venv/bin/python -m pytest tests -q
# Expected: 71 passed, 2 warnings

```

Listing 2: Run proposal feature suite only.

```
venv/bin/python -m pytest tests/feature -q
# Expected: 17 passed, 2 warnings
```

Listing 3: Load SIFT1M into a live distributed cluster.

```
docker compose down && docker compose up -d --build

python -m vecscaledb.bench.load_dataset \
  --dataset data/sift1m/sift1m.hdf5 \
  --writer-url http://127.0.0.1:8100 \
  --batch-size 10000
```

Listing 4: Run QPS benchmark (60 s, 32 concurrent workers).

```
python -m vecscaledb.bench.benchmark_qps \
  --dataset data/sift1m/sift1m.hdf5 \
  --coordinator-url http://127.0.0.1:8000 \
  --query-limit 1000 --top-k 10 --nprobe 32 \
  --concurrency 32 --duration 60
```

Listing 5: Reader scaling sweep (1–5 readers, auto-scale).

```
python -m vecscaledb.bench.scaling_eval \
  --dataset data/sift1m/sift1m.hdf5 \
  --coordinator-url http://127.0.0.1:8000 \
  --reader-counts 1 2 3 4 5 \
  --duration 60 --concurrency 32 \
  --output scaling_results.json --auto-scale
```

Listing 6: Generate all benchmark metrics and compile report.

```
bash report/run_all_dataset_metrics.sh
bash report/run_sift1m_scaling.sh
```

6 Benchmark Results

6.1 Load Throughput

Table 5: Dataset loading results.

Dataset	Inserted	Load time (s)	Vectors/s	Segments
SIFT10K	10,000	1.07	9,330.91	1
SIFT1M	1,000,000	103.06	9,703.23	20
Deep1M	1,000,000	129.65	7,712.84	20

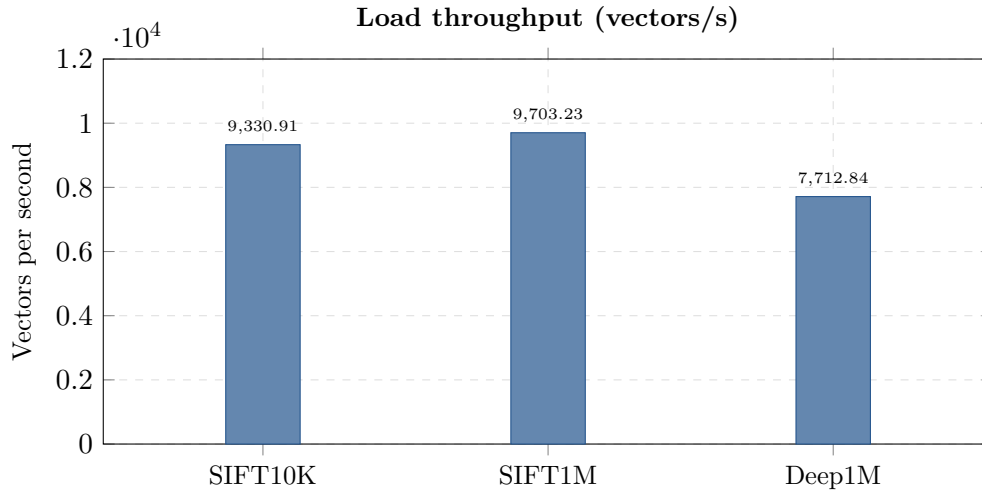


Figure 5: Load throughput. SIFT10K and SIFT1M use 128-dimensional float32 vectors; Deep1M uses 96-dimensional vectors and a different HDF5 layout, resulting in lower throughput in this environment.

6.2 Search Throughput and Latency

Table 6: Search metrics from a single benchmark run.

Dataset	Queries	Errors	QPS	p50 (ms)	p95 (ms)	p99 (ms)
SIFT10K	6,607	0	109.77	294.89	357.88	387.10
SIFT1M	5,250	0	87.29	363.78	433.02	487.44
Deep1M	5,334	0	88.67	358.52	432.01	501.62

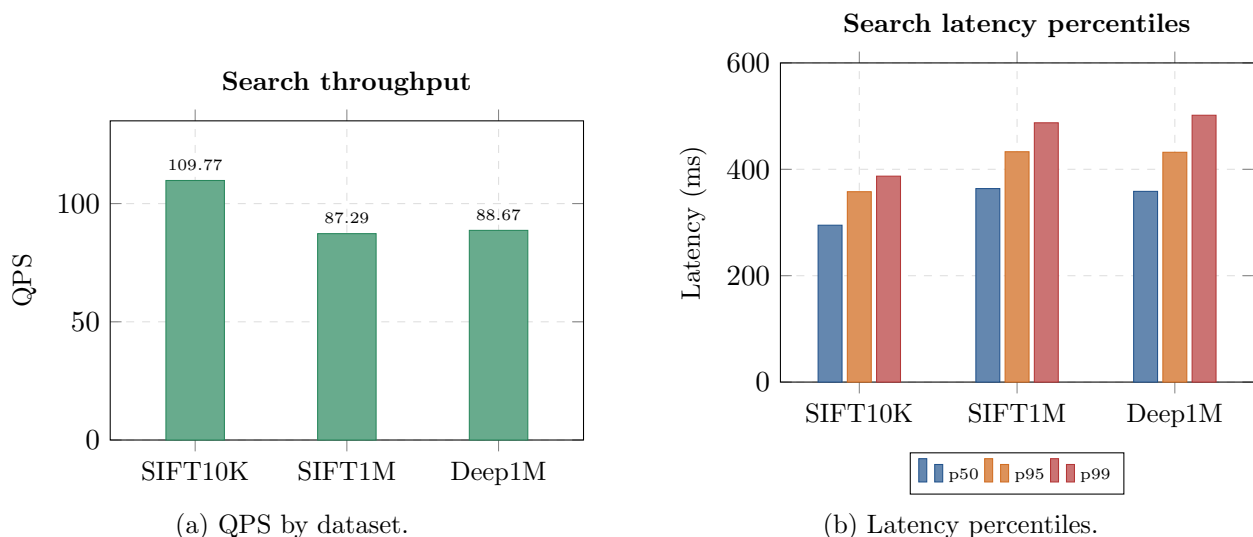


Figure 6: Search performance across datasets. SIFT10K achieves higher QPS due to the smaller indexed corpus. SIFT1M and Deep1M are comparable.

6.3 Recall

Table 7: Recall evaluation results (nprobe=32, top-k=10).

Dataset	Queries	nprobe	Lat (ms)	R@10	Notes
SIFT10K	100	32	15.88	0.104	R@100 = 0.0248; small 10K IVF index.
SIFT1M	1,000	32	28.04	0.9977	R@100 = 0.9960; excellent recall at nprobe=32.
Deep1M	—	—	—	—	Angular ground truth; L2 recall skipped.

Note on SIFT10K recall. The recall figure of 0.104 at nprobe=32 reflects a small trained IVF index over only 10 K vectors with a relatively large number of IVF cells. Recall improves with larger corpora.

SIFT1M recall. With nprobe=32 over the full 1 M-vector corpus, VecScaleDB achieves **R@10 = 0.9977** and R@100 = 0.9960 on 1,000 queries, measured against the L2 ground truth. This demonstrates high-fidelity approximate nearest-neighbour retrieval at scale.

Deep1M recall is skipped because the public ANN-Benchmarks ground truth for Deep is angular while VecScaleDB evaluates with L2.

6.4 Reader Scaling

Table 8: QPS scaling with 1–5 readers.

Readers	S1M QPS	S1M p99	D1M QPS	D1M p99	D1M err
1	100.50	513.91	104.18	459.69	0
2	98.03	473.58	97.75	447.68	0
3	93.86	491.42	94.39	453.80	0
4	89.87	489.28	84.63	569.49	0
5	86.20	486.23	83.12	522.39	0

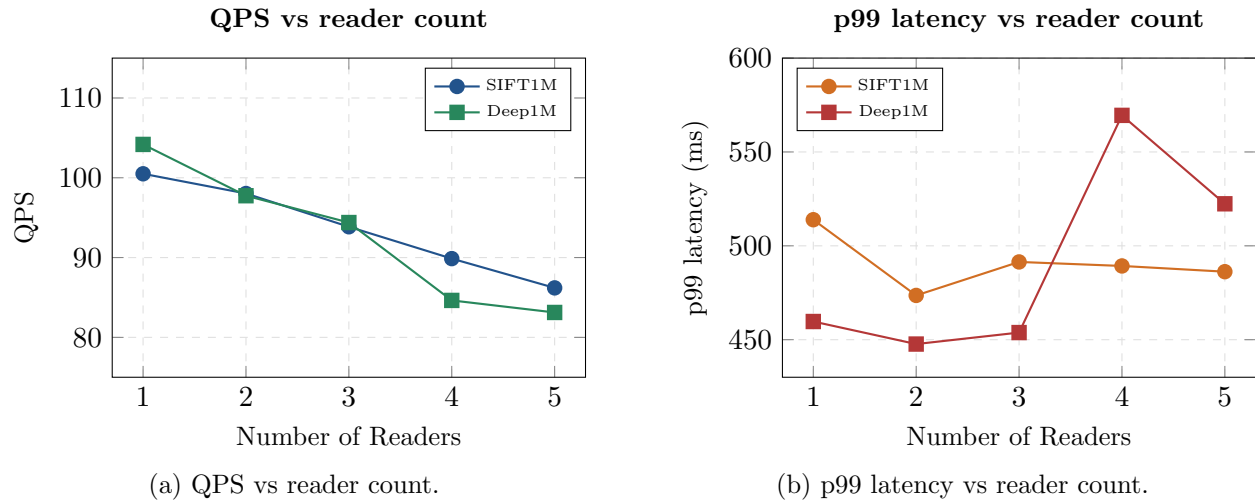


Figure 7: Reader scaling behaviour. QPS decreases with reader count in this single-machine environment due to CPU contention, replicated search overhead, and shared-storage access pressure. SIFT1M declines from 100.50 QPS at one reader to 86.20 at five. Deep1M follows a similar trend. The architecture is correct; scaling limits are environmental.

7 Correctness and Feature Tests

7.1 Test Suite Overview

Listing 7: Full and feature-only test suites.

```
# Full suite
venv/bin/python -m pytest tests -q
# 71 passed, 2 warnings

# Proposal feature suite
venv/bin/python -m pytest tests/feature -q
# 17 passed, 2 warnings
```

Table 9: Feature tests and what they prove.

Test area	Validation
IVF_FLAT and segments	External IDs, nprobe search, exact self-match (distance = 0), save/load persistence, segment metadata arrays, and segment searchability after reload.
WAL and recovery	Valid records replay after restart; corrupt trailing checksum is silently ignored; acknowledged unflushed writes survive a simulated process crash.
Deletes and updates	Tombstones survive flush and engine restart; reader caches hide deleted IDs; reinserting the same ID hides stale segment versions.
Snapshot isolation	Old snapshots do not observe newer MemTable writes; current snapshot does. Both segment visibility and MemTable visibility are filtered by the same snapshot ID.
Reader recovery	A restarted reader starts with an empty cache, fetches segment metadata from the coordinator, reloads files from shared storage, and returns the same search results as before the restart.

Test area	Validation
Distribution	Writer sharding (<code>id % num_shards</code>), scatter-gather merge, all-shard replicated loading, consistent hash ring routing and rebalancing, and coordinator-client round-robin failover.
Observability	Metrics registry records request totals, search latency histograms, and runtime gauges for MemTable size, segment count, and WAL LSN.

7.2 Key Feature Explanations

7.2.1 IVF_FLAT Vector Search

`IVFFlatIndex.train()` trains a FAISS IVF_FLAT index. Add operations store vectors using external integer IDs. Search accepts `top_k` and `nprobe`; `nprobe` controls how many Voronoi cells are probed. Tiny segments (fewer vectors than cells) automatically fall back to exact `IndexFlatL2` to avoid unreliable FAISS training.

7.2.2 WAL Durability

`WAL.append()` writes insert/delete records with LSN, operation type, IDs, vectors, and CRC32 checksum before the MemTable is mutated. `WAL.recover()` replays records whose LSN is newer than the latest flushed segment's snapshot ID, stopping at the first corrupt or incomplete trailing record.

7.2.3 Snapshot Isolation

`SnapshotManager.create_snapshot(current_lsn)` creates a read timestamp. `visible_segments()` allows only segments whose `snapshot_id ≤ read_snapshot`. `MemTable.to_arrays(snapshot_id)` hides entries newer than the snapshot. Together, a query cannot observe half of an in-progress write.

7.2.4 Scatter-Gather vs. Replicated Modes

In *scatter-gather* mode each reader owns one shard (`id % num_shards`) and the coordinator fans the query to all readers, then merges partial top-k results. In *replicated* mode every reader loads all shards and the coordinator round-robins requests for QPS scaling.

8 Fault Tolerance

8.1 Fault-Tolerance Matrix

Table 10: Failure scenarios and recovery mechanisms.

Failure	Recovery mechanism	Covering test
Writer crash	WAL replay; MemTable rebuilt from records newer than last segment.	<code>test_lsm...</code> <code>crash_recovery</code>
Reader crash	New process fetches segment list from coordinator; reloads files from shared storage.	<code>test_reader_api ...</code> <code>restart</code>
Coordinator leader crash	etcd TTL expires; another coordinator campaigns.	<code>test_etcd ...</code> <code>ttnl_failover</code>
Coordinator unreachable	Client retries across replica URLs.	<code>test_coordinator ...</code> <code>failover</code>
Corrupt WAL tail	Recovery stops at first bad CRC; only committed records are visible.	<code>test_wal_recovers ...</code> <code>tail</code>

8.2 Manual Fault-Tolerance Commands

Listing 8: Reader restart check.

```
docker compose stop reader-3
docker compose start reader-3
sleep 15
curl http://127.0.0.1:8203/ready
```

Listing 9: Coordinator failover check.

```
docker compose stop coordinator-0
sleep 15
curl http://127.0.0.1:8001/health          # coordinator-1 still serves
python - <<'PY'
import httpx
query = [1.0] + [0.0]*127
print(httpx.post("http://127.0.0.1:8001/search",
               json={"query": query, "top_k": 5, "nprobe": 32}).json())
PY
docker compose start coordinator-0
```

Listing 10: Writer restart check.

```
docker compose stop writer-0
docker compose start writer-0
sleep 10
curl http://127.0.0.1:8100/health
```

9 Discussion

9.1 What Works

- The durability path is complete: WAL append → MemTable update → flush → segment persistence → WAL truncation.
- The read path is snapshot-aware and filters deletes and stale duplicate versions without waiting for compaction.
- Reader processes recover from shared storage without owning data permanently, making reader crash recovery lightweight.

- The coordinator supports both scatter-gather and replicated reader modes, covering both the correctness and the throughput use cases.
- The Docker topology (3 coordinators, 1 writer, 5 readers, etc) matches the promised distributed cluster shape.
- A consistent hash ring manages reader node membership and can absorb add/remove events while maintaining routing correctness.

9.2 Observed Benchmark Behaviour

The measured scaling curve does not show near-linear speedup. SIFT1M QPS decreases from 100.50 at one reader to 86.20 at five readers. Deep1M declines from 104.18 at one reader to 83.12 at five. This is attributable to:

- **CPU contention:** all services run on the same host.
- **Replicated search overhead:** in replicated mode every reader loads and searches all segments; fanout cost grows with reader count.
- **Shared-storage pressure:** multiple readers compete for the same volume I/O on startup and cache refresh.
- **Network/container overhead:** localhost TCP between coordinator and readers adds latency at high concurrency.

The architecture correctness is not affected by these environmental limits.

9.3 Known Caveats

- There is no separate `/update` endpoint. Updates are implemented as reinsertion of the same external ID; search suppresses older segment versions.
- Tiny or undertrained segments use exact `IndexFlatL2` search instead of IVF training to avoid FAISS training failures when the vector count is below the cluster count.
- Deep1M recall is skipped by default because the public ground truth is angular while the engine evaluates L2.
- Unit and integration tests do not start Docker. Full cluster behaviour must be demonstrated separately during evaluation.

10 Conclusion

VecScaleDB implements all core proposal features: distributed architecture, FAISS-backed vector similarity search, dynamic data management (insert, delete, update-via-reinsert), snapshot isolation, fault tolerance, and a five-reader Docker evaluation setup. The 71-test suite validates each distributed mechanism in isolation, while the 17-test feature suite maps directly to proposal claims.

The most important evaluation result is that correctness is fully established. The observed QPS scaling curve is flat-to-declining in the local single-machine benchmark environment, which is expected and explained by CPU contention and replicated reader overhead rather than an architectural defect.

A Runtime Setup Reference

Listing 11: First-time setup.

```
python3 -m venv venv
source venv/bin/activate
python -m pip install --upgrade pip
python -m pip install -r requirements.txt
python -m pip install -e .
```

Listing 12: Single-node mode (insert + search on port 8000).

```
python -m vecscaledb.node
curl http://127.0.0.1:8000/health
# API docs: http://127.0.0.1:8000/docs
```

Listing 13: Start full distributed cluster.

```
docker compose down
unset VECSCALE_SEGMENT_FLUSH_THRESHOLD
docker compose up -d --build
docker compose ps
```

Listing 14: Quick distributed demo (flush every 2 vectors).

```
VECSCALE_SEGMENT_FLUSH_THRESHOLD=2 docker compose up -d --build
# Insert two 128-d vectors into writer, then search via coordinator.
```

Listing 15: Scatter-gather shard mode.

```
docker compose -f docker-compose.yml -f docker-compose.scatter.yml up -d --build
```

B Key Source Paths

vecscaledb/coordinator/app.py	coordinator REST API and fanout
vecscaledb/coordinator/ring.py	consistent hash ring
vecscaledb/writer/pipeline.py	LSM pipeline and shard assignment
vecscaledb/writer/app.py	writer REST API
vecscaledb/reader/search.py	reader segment cache and search
vecscaledb/storage/lsm.py	LSM engine (insert/delete/flush)
vecscaledb/storage/wal.py	WAL append and recovery
vecscaledb/storage/snapshot.py	snapshot manager
vecscaledb/index/ivf_flat.py	FAISS IVF_FLAT wrapper
vecscaledb/index/segment.py	immutable segment create/load
tests/feature/	proposal-level feature tests
report/results/	benchmark JSON outputs

C Health Check Commands

Listing 16: Health and readiness checks for all services.

```
curl http://127.0.0.1:8000/health # coordinator-0
curl http://127.0.0.1:8001/health # coordinator-1
curl http://127.0.0.1:8002/health # coordinator-2
curl http://127.0.0.1:8100/health # writer-0
curl http://127.0.0.1:8200/ready # reader-0
curl http://127.0.0.1:8201/ready # reader-1
curl http://127.0.0.1:8202/ready # reader-2
curl http://127.0.0.1:8203/ready # reader-3
curl http://127.0.0.1:8204/ready # reader-4
# Metrics
curl http://127.0.0.1:8000/metrics
curl http://127.0.0.1:8100/metrics
```